JAVA PROGRAMMING / DSA

Chapter 45: Advanced Graph Algorithms

Personal Notes

Chapter 33 covered the graph foundations: DFS, BFS, topological sort, and Dijkstra for shortest paths in weighted graphs. That toolkit handles maybe 70% of graph problems. This chapter tackles the remaining 30% — the algorithms that come up when Dijkstra fails, when you need distances between EVERY pair, when the graph has cycles or negative weights, and when you need to find the structural weak points of a network.

Five advanced algorithms round out the graph toolkit: Bellman-Ford (Dijkstra's negative-weight cousin), Floyd-Warshall (all-pairs shortest paths), Prim's MST (dense-graph alternative to Kruskal's from Ch 37), and Tarjan's algorithms for articulation points, bridges, and strongly connected components. Each solves a specific class of problems Dijkstra can't — and each shows up in enough Google interviews to be worth knowing.

1. Recap — What Ch 33 & 37 Already Covered

Algorithm	What It Does
DFS / BFS	Traversal, cycle detection, connected components
Topological Sort	Ordering DAG nodes (Kahn's or DFS)
Dijkstra	Shortest path with NON-NEGATIVE weights
Union-Find (Ch 37)	Fast connectivity, cycle detection while adding edges
Kruskal's MST (Ch 37)	MST by sorting edges + DSU

This chapter fills in the gaps: negative weights, all-pairs distances, MST for dense graphs, and structural analysis (weak points and strongly connected components).

2. Bellman-Ford — Shortest Path with Negative Weights

Dijkstra fails on graphs with negative edges. Bellman-Ford handles them — AND detects negative cycles as a bonus. The trade-off: O(V × E) instead of Dijkstra's O((V+E) log V). Slower, but universal.

The Key Idea

Relax every edge V-1 times. After V-1 iterations, all shortest paths are guaranteed to be correct — unless a negative cycle exists, in which case a Vth pass will still improve some distance (proving the negative cycle).

```
public int[] bellmanFord(int n, int[][] edges, int source) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[source] = 0;

    // Relax edges V-1 times
    for (int i = 0; i < n - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    // One more pass — if anything still improves, we have a negative
cycle
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], w = edge[2];
        if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
            return null;   // negative cycle detected
        }
    }

    return dist;
}

// Time: O(V × E)
// Space: O(V)
```

Why V-1 Iterations?

```
     After iteration 1: shortest paths using at most 1 edge are correct.
     After iteration 2: shortest paths using at most 2 edges are correct.
     ...
     After iteration V-1: shortest paths of ANY length ≤ V-1 edges are
correct.

     (Any simple path in a graph has at most V-1 edges.)

     If a Vth pass STILL relaxes something → there must be a negative
cycle
     reachable from the source, since a longer path shouldn't be shorter.
```

When to use Bellman-Ford: Use it whenever you need shortest path AND (a) negative edges exist, or (b) you need to detect negative cycles. For non-negative weights, Dijkstra is strictly better (faster). Bellman-Ford's O(V × E) is fine for graphs with a few thousand nodes and edges — but too slow for millions.

3. Floyd-Warshall — All-Pairs Shortest Paths

When you need the shortest path between EVERY pair of nodes, running Dijkstra from each source works but costs O(V × (V+E) log V). Floyd-Warshall does it in $O(V^3)$ with beautifully simple code — usually the right choice for dense graphs with V ≤ ~500.

The Idea

Consider intermediate nodes one at a time. For each candidate midpoint k, check every pair (i, j) — is going THROUGH k shorter than the current best? Update if so. After considering all V nodes as potential midpoints, all shortest paths are found.

```
public int[][] floydWarshall(int[][] graph) {
    int n = graph.length;
    int[][] dist = new int[n][n];

    // Initialize with direct edges
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];   // Integer.MAX_VALUE / 2 if no
edge
        }
    }

    // Try each node as an intermediate
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    return dist;
}

// Time: O(V³)
// Space: O(V²)
```

> **Use MAX_VALUE / 2 for infinity:** If you use Integer.MAX_VALUE directly and then compute dist[i][k] + dist[k][j], it overflows to a NEGATIVE number — making unreachable pairs look shorter than they are. Use MAX_VALUE / 2 (or Long) as your "infinity" sentinel to avoid this trap.

Detecting Negative Cycles

After Floyd-Warshall, check dist[i][i] for any i. A negative value there means i can reach itself through a negative-cost loop — a negative cycle.

4. Prim's Algorithm — MST for Dense Graphs

You already learned Kruskal's MST (Ch 37) — sort edges, use DSU. PRIM'S is the alternative: grow the MST one node at a time, always adding the CHEAPEST edge crossing the frontier. Uses a min-heap, similar to Dijkstra's spirit.

The Algorithm

1. Start from any node — add it to the "in MST" set
2. Among all edges connecting an in-MST node to an out-of-MST node, pick the CHEAPEST
3. Add its out-of-MST endpoint to the MST
4. Repeat until all nodes are in the MST

```
public int primMST(int n, List<List<int[]>> graph) {
    // graph[u] = list of {v, weight}
    boolean[] inMST = new boolean[n];
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{0, 0});   // start from node 0 with cost 0

    int totalCost = 0, nodesAdded = 0;

    while (!pq.isEmpty() && nodesAdded < n) {
        int[] curr = pq.poll();
        int u = curr[0], w = curr[1];

        if (inMST[u]) continue;
        inMST[u] = true;
        totalCost += w;
        nodesAdded++;

        for (int[] edge : graph.get(u)) {
            int v = edge[0], ew = edge[1];
            if (!inMST[v]) pq.offer(new int[]{v, ew});
        }
    }

    return nodesAdded == n ? totalCost : -1;   // -1 if disconnected
}

// Time: O(E log V) with binary heap
```

Prim vs Kruskal

Aspect	Prim's	Kruskal's (Ch 37)
Data structure	Min-heap (like Dijkstra)	DSU (Union-Find)
Time complexity	O(E log V)	O(E log E)

Aspect	Prim's	Kruskal's (Ch 37)
Best for	DENSE graphs (E close to V^2)	SPARSE graphs (E much less than V^2)
Order of adding	Grows one MST from a start node	Picks globally-cheapest edges
Ease of coding	Slightly more setup (adjacency list)	Simpler if edges are already given

Both produce a valid MST: Kruskal's and Prim's always give a minimum spanning tree with the same total weight. They differ only in HOW they build it. In interviews, use Kruskal's if edges are given as a list and DSU is natural; use Prim's if the graph is dense or already given as an adjacency list.

5. Articulation Points (Cut Vertices)

An ARTICULATION POINT is a node whose removal DISCONNECTS the graph. Real-world: bridges in road networks, critical routers on a network, key employees in an organization. Tarjan's algorithm finds them all in O(V + E) via DFS.

The Insight

During DFS, track two values for each node u: disc[u] = when u was first visited, low[u] = the earliest disc reachable from u's subtree via back edges. Node u is an articulation point if it has a child v where low[v] ≥ disc[u] — meaning v's subtree has no back edge escaping u.

```
class ArticulationPoints {
    int[] disc, low;
    boolean[] visited, isArt;
    int timer = 0;
    List<List<Integer>> graph;

    public List<Integer> findArticulationPoints(int n, List<List<Integer>>
graph) {
        this.graph = graph;
        disc = new int[n];
        low = new int[n];
        visited = new boolean[n];
        isArt = new boolean[n];

        for (int i = 0; i < n; i++) {
            if (!visited[i]) dfs(i, -1);
        }

        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < n; i++) if (isArt[i]) result.add(i);
        return result;
    }
```

```
    private void dfs(int u, int parent) {
        visited[u] = true;
        disc[u] = low[u] = timer++;
        int children = 0;

        for (int v : graph.get(u)) {
            if (!visited[v]) {
                children++;
                dfs(v, u);
                low[u] = Math.min(low[u], low[v]);

                // Non-root: u is articulation if any child can't escape
past u
                if (parent != -1 && low[v] >= disc[u]) isArt[u] = true;
            } else if (v != parent) {
                low[u] = Math.min(low[u], disc[v]);
            }
        }

        // Root is articulation if it has more than one DFS-tree child
        if (parent == -1 && children > 1) isArt[u] = true;
    }
}

// Time: O(V + E)
```

Root is special-cased: The general rule fails for the DFS root because it has no parent. Instead: the root is an articulation point IFF it has more than one child in the DFS tree. That's an unrelated condition — always handle roots separately.

6. Bridges (Cut Edges)

A BRIDGE is an edge whose removal disconnects the graph. Tarjan's algorithm finds bridges with the same DFS as articulation points — just a slightly different condition. If disc[u] < low[v] (STRICT inequality) for an edge u-v where v is u's child, then u-v is a bridge.

```
// Same DFS as articulation points — but capture bridges instead:
if (low[v] > disc[u]) bridges.add(new int[]{u, v});

// Explanation: if v's subtree cannot reach anything at or before u's
disc,
// then removing edge u-v isolates v's subtree.

// Time: O(V + E)
```

Articulation vs Bridge: An articulation point is a NODE whose removal disconnects. A bridge is an EDGE whose removal disconnects. Related but distinct. In dense graphs, articulation points are more

common than bridges. Both use the same low/disc DFS framework — memorize the framework once, adapt the condition.

7. Strongly Connected Components (SCCs)

In a DIRECTED graph, a STRONGLY CONNECTED COMPONENT (SCC) is a maximal group of nodes where every node can reach every other node. Two well-known algorithms: Kosaraju's (two DFS passes) and Tarjan's (single DFS with a stack).

Kosaraju's Algorithm — Simpler to Understand

5. Run DFS on the original graph, pushing nodes onto a stack when done
6. Reverse all edges to get the transpose graph
7. Pop nodes from the stack. Each unvisited pop starts a new DFS in the transpose — that DFS's visited set is one SCC

```
public List<List<Integer>> kosaraju(int n, List<List<Integer>> graph) {
    Deque<Integer> stack = new ArrayDeque<>();
    boolean[] visited = new boolean[n];

    // Pass 1: fill stack in order of finish time
    for (int i = 0; i < n; i++) {
        if (!visited[i]) fillOrder(i, visited, stack, graph);
    }

    // Build transpose graph
    List<List<Integer>> reverse = new ArrayList<>();
    for (int i = 0; i < n; i++) reverse.add(new ArrayList<>());
    for (int u = 0; u < n; u++)
        for (int v : graph.get(u))
            reverse.get(v).add(u);

    // Pass 2: DFS the transpose in stack order
    Arrays.fill(visited, false);
    List<List<Integer>> sccs = new ArrayList<>();
    while (!stack.isEmpty()) {
        int u = stack.pop();
        if (!visited[u]) {
            List<Integer> component = new ArrayList<>();
            collectSCC(u, visited, reverse, component);
            sccs.add(component);
        }
    }
    return sccs;
}

private void fillOrder(int u, boolean[] visited, Deque<Integer> stack,
                       List<List<Integer>> graph) {
```

```
    visited[u] = true;
    for (int v : graph.get(u)) {
        if (!visited[v]) fillOrder(v, visited, stack, graph);
    }
    stack.push(u);
}

private void collectSCC(int u, boolean[] visited, List<List<Integer>>
reverse,
                        List<Integer> component) {
    visited[u] = true;
    component.add(u);
    for (int v : reverse.get(u)) {
        if (!visited[v]) collectSCC(v, visited, reverse, component);
    }
}

// Time: O(V + E)
```

Tarjan's SCC (also O(V + E)) is a single-pass algorithm that's more elegant but harder to code. For interviews, Kosaraju's is often enough.

8. Classic Problem 1 — Cheapest Flights Within K Stops

LeetCode 787. Find the cheapest flight from src to dst using at most K stops. Since Dijkstra doesn't handle the K-stops constraint naturally, MODIFIED BELLMAN-FORD is the clean approach.

```
public int findCheapestPrice(int n, int[][] flights, int src, int dst, int
k) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Relax K+1 times (each iteration adds one more edge to allowed
paths)
    for (int i = 0; i <= k; i++) {
        int[] prev = dist.clone();
        for (int[] f : flights) {
            int u = f[0], v = f[1], w = f[2];
            if (prev[u] == Integer.MAX_VALUE) continue;
            dist[v] = Math.min(dist[v], prev[u] + w);
        }
    }

    return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
}

// Time: O(K × E)
```

The clone() trick is crucial: Without it, updates within one iteration can 'chain' — using an already-updated distance in the same round. That would count more than one edge per round. Cloning prev = dist at the start of each iteration keeps each round strictly one-edge.

9. Classic Problem 2 — Network Delay Time

LeetCode 743. Given a network of nodes and directed weighted edges, find the time it takes for a signal from source K to reach ALL nodes. Dijkstra works — but it's a solid warm-up for shortest-path variants.

```
public int networkDelayTime(int[][] times, int n, int k) {
    List<List<int[]>> graph = new ArrayList<>();
    for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());
    for (int[] t : times) graph.get(t[0]).add(new int[]{t[1], t[2]});

    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{k, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int u = curr[0], d = curr[1];
        if (d > dist[u]) continue;

        for (int[] edge : graph.get(u)) {
            int v = edge[0], w = edge[1];
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.offer(new int[]{v, dist[v]});
            }
        }
    }

    int max = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == Integer.MAX_VALUE) return -1;
        max = Math.max(max, dist[i]);
    }
    return max;
}
```

10. Classic Problem 3 — Critical Connections in a Network

LeetCode 1192. Find all critical connections (bridges) in a network of servers. Direct application of the bridge-finding algorithm from Section 6.

```
class CriticalConnections {
    int[] disc, low;
    int timer = 0;
    List<List<Integer>> graph;
    List<List<Integer>> bridges = new ArrayList<>();

    public List<List<Integer>> criticalConnections(int n,
List<List<Integer>> connections) {
        graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        for (List<Integer> c : connections) {
            graph.get(c.get(0)).add(c.get(1));
            graph.get(c.get(1)).add(c.get(0));
        }

        disc = new int[n];
        low  = new int[n];
        Arrays.fill(disc, -1);

        for (int i = 0; i < n; i++) {
            if (disc[i] == -1) dfs(i, -1);
        }
        return bridges;
    }

    private void dfs(int u, int parent) {
        disc[u] = low[u] = timer++;
        for (int v : graph.get(u)) {
            if (v == parent) continue;
            if (disc[v] == -1) {
                dfs(v, u);
                low[u] = Math.min(low[u], low[v]);
                if (low[v] > disc[u]) {
                    bridges.add(Arrays.asList(u, v));
                }
            } else {
                low[u] = Math.min(low[u], disc[v]);
            }
        }
    }
}
```

11. Classic Problem 4 — Redundant Connection II

LeetCode 685. Given a directed graph that would be a tree if one edge were removed, find that redundant edge. Tricky variant — three cases (extra parent, cycle, or both). A union-find + special-case approach handles all three.

This problem is more of a case-analysis exercise than a new algorithm, but it's a Google interview classic that showcases how DSU handles directed-graph edge cases. Worth solving once for the trickiness.

12. When to Reach for Which Algorithm

If you need to...	Use
Shortest path (non-negative weights)	**Dijkstra (Ch 33)**
Shortest path (negative weights allowed)	**Bellman-Ford**
Detect negative cycles	**Bellman-Ford**
All-pairs shortest paths (V ≤ ~500)	**Floyd-Warshall**
All-pairs shortest paths (large V)	**Dijkstra from each node**
MST — sparse graph, edges as a list	**Kruskal's (Ch 37)**
MST — dense graph, adjacency list	**Prim's**
Find critical vertices	**Articulation Points (Tarjan)**
Find critical edges	**Bridges (Tarjan)**
Strongly connected groups (directed)	**Kosaraju's or Tarjan's SCC**
Constrained shortest path (K stops)	**Modified Bellman-Ford**

13. Complexity Summary

Algorithm	Time	Space
Dijkstra (Ch 33)	O((V + E) log V)	O(V)
Bellman-Ford	O(V × E)	O(V)
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Prim's MST (heap)	O(E log V)	O(V + E)
Kruskal's MST (Ch 37)	O(E log E)	O(V + E)
Articulation Points	O(V + E)	O(V + E)
Bridges	O(V + E)	O(V + E)
Kosaraju's SCC	O(V + E)	O(V + E)

14. Common Mistakes

Mistake 1: Using Dijkstra with negative edges

Dijkstra can give wrong results if any edge has negative weight — it assumes visiting the closest unvisited node means we've found the shortest path there. With negatives, a later path might be shorter. Always use Bellman-Ford when negatives exist.

Mistake 2: Using Integer.MAX_VALUE in Floyd-Warshall

```
dist[i][k] + dist[k][j]     // ✗ overflows if either is MAX_VALUE

// Fix: use a smaller sentinel, or check for MAX_VALUE before adding:
if (dist[i][k] != INF && dist[k][j] != INF) {
    dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
}
```

Mistake 3: Wrong loop order in Floyd-Warshall

```
// The outermost loop MUST be k (the intermediate).
for (int k = 0; k < n; k++)      // ✓ correct
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) ...

// Any other order gives wrong results — even though the code looks the
same.
```

Mistake 4: Forgetting root special case in articulation points

The general rule (low[v] ≥ disc[u]) fails for the DFS root. Root is articulation ⟺ it has more than one DFS-tree child. Always handle this case explicitly.

Mistake 5: Not cloning dist in K-stop shortest path

If you update dist in-place during Bellman-Ford iterations, you accidentally use more than one edge per iteration. Clone at the start of each round to enforce the K-stop constraint.

Mistake 6: Confusing Prim's with Dijkstra

They look almost identical. The difference: Dijkstra tracks TOTAL DISTANCE from the source (accumulated); Prim's tracks the WEIGHT OF THE SINGLE EDGE that connects to the MST. The heap stores different things.

Mistake 7: Skipping DSU tricks with SCC problems

SCC problems in directed graphs are NOT DSU problems — DSU can't handle directed edges properly (it collapses direction). Use Kosaraju's or Tarjan's SCC algorithms instead.

15. Best Practices

- Try Dijkstra first for shortest path — only use Bellman-Ford if negatives exist
- Use Floyd-Warshall for small dense graphs (V ≤ 500 or so)
- For Floyd-Warshall, initialize infinity as MAX_VALUE / 2 (or use long)
- Prim's for dense graphs with adjacency lists; Kruskal's for sparse or edge-list input
- Handle root separately in articulation-point algorithms — it's a special case
- For K-stop shortest path, clone dist array at each iteration to enforce edge count
- Learn the low/disc DFS framework once — it handles articulation points, bridges, and Tarjan's SCC
- SCC needs directed-graph specific algorithms — DSU/BFS won't work

16. Quick Reference

Concept	Meaning
Bellman-Ford	Shortest path with negative edges + negative cycle detection
Floyd-Warshall	All-pairs shortest paths in $O(V^3)$
Prim's MST	Grow MST from one node using a min-heap
Articulation Point	Node whose removal disconnects the graph
Bridge	Edge whose removal disconnects the graph
disc[]	Discovery time in DFS
low[]	Earliest disc reachable via back edges
SCC	Strongly Connected Component (directed graph)
Kosaraju's Algorithm	Two DFS passes: original then transposed graph
Negative cycle	Bellman-Ford relaxes a Vth time = negative cycle
V-1 rule	Simple paths have ≤ V-1 edges → Bellman-Ford iterates V-1 times

17. Practice Problems

Try in order — Dijkstra-style first, then Bellman-Ford, then structural analysis.

Shortest Path

8. Network Delay Time (LeetCode 743) — Dijkstra warm-up.

9. Path with Maximum Probability (LeetCode 1514) — modified Dijkstra.
10. Cheapest Flights Within K Stops (LeetCode 787) — modified Bellman-Ford.
11. Minimum Cost to Reach Destination in Time (LeetCode 1928) — Dijkstra with time constraint.
12. Detect negative cycle in a directed weighted graph — Bellman-Ford.

All-Pairs

13. Find the City With the Smallest Number of Neighbors at a Threshold Distance (LeetCode 1334) — Floyd-Warshall.
14. Course Schedule IV (LeetCode 1462) — reachability closure, Floyd-Warshall variant.

Minimum Spanning Tree

15. Min Cost to Connect All Points (LeetCode 1584) — Prim's or Kruskal's.
16. Connecting Cities With Minimum Cost (LeetCode 1135).
17. Optimize Water Distribution in a Village (LeetCode 1168).

Structural Analysis

18. Critical Connections in a Network (LeetCode 1192) — bridges.
19. Number of Distinct Islands (LeetCode 694).
20. Redundant Connection II (LeetCode 685) — the tricky directed case.

SCC & Advanced

21. Find Eventual Safe States (LeetCode 802) — reverse graph + topological.
22. Longest Cycle in a Graph (LeetCode 2360).
23. Count Strongly Connected Components — Kosaraju's.
24. Reconstruct Itinerary (LeetCode 332) — Hierholzer's algorithm (Eulerian path).

18. Final Takeaway

Advanced graph algorithms fill in the gaps that Ch 33's toolkit couldn't handle: negative weights (Bellman-Ford), all-pairs paths (Floyd-Warshall), dense MSTs (Prim's), and structural analysis (articulation points, bridges, SCCs). Each is more specialized than DFS or Dijkstra — but each unlocks a class of problems the basics can't touch.

For interviews, master three algorithms: Bellman-Ford (with negative cycle detection), Floyd-Warshall (all-pairs), and the low/disc DFS framework (for articulation points AND bridges). Prim's and Kosaraju's are useful bonuses. Together with Ch 33 and Ch 37, this completes your graph toolkit — every major graph algorithm you'd encounter in a Google interview is now covered.

> **Key Takeaway:** Dijkstra fails on negatives → use BELLMAN-FORD. Need distances between every pair? FLOYD-WARSHALL in $O(V^3)$. MST for dense graphs → PRIM'S with a min-heap. Find weak spots

> in a network → low/disc DFS gives you articulation points and bridges from the same framework. For directed connectivity groups → KOSARAJU'S with two DFS passes. Each algorithm targets a specific case Dijkstra can't handle — learn the signals and pick the right tool.

19. What's Next?

With advanced graphs done, the DSA toolkit is essentially complete. Remaining specialized topics:

- KMP / Rabin-Karp — string pattern matching in O(n + m)
- Number Theory basics — modular arithmetic, primes, GCD tricks, combinatorics
- Randomized Algorithms — Quickselect, Reservoir Sampling
- Suffix Arrays / Automata — for advanced string problems
- Max Flow / Min Cut — Ford-Fulkerson, Edmonds-Karp
- Computational Geometry — convex hull, line sweep, closest pair
- System Design fundamentals — the natural big step for senior interviews